



Université Joseph Ki-ZERBO



Unité de formation et de recherche en
sciences exactes et appliquées
(UFR/SEA)

RAPPORT DE PROJET

LandGuard Neuro-Symbolic AI Détection intelligente des fraudes foncières

Tronc commun Master 1 Informatique
IA Symbolique, Probabiliste & Neuro-Symbolique

Membres du groupe 1 :

DAKOURE Wendlasida Lesly Laurine

GNISSIEN Migouel

OUATTARA Barkissa

Encadrant :

Dr. Cédric BERE

Année Académique 2025–2026

Table des matières

Introduction	1
0.1 Objectifs	2
0.2 Principe éthique	2
1 Contexte et problématique	3
1.1 Limites des approches isolées	3
2 Architecture générale du système	3
3 Modélisation en logique de description	3
3.1 Taxonomie	3
3.2 Rôles	3
3.3 Axiomes métier	4
3.4 Contraintes d'intégrité	5
4 Base de connaissances Prolog	5
4.1 Exemples de faits	5
5 Raisonnement symbolique avec Prolog	5
5.1 Organisation des règles	5
5.2 Exemple de règle	6
5.3 Moteur d'inférence	7
6 Explicabilité XAI	7
7 Raisonnement probabiliste avec ProbLog	7
7.1 Clauses probabilistes	7
7.2 Résultats observés	8
7.3 Échelle de criticité	8
8 Couche neuronale et neuro-symbolique	8
8.1 Modèle PyTorch	8
8.2 Spécification DeepProbLog	9
9 Dataset et analyse expérimentale	9
9.1 Structure du dataset	9
9.2 Colonnes principales	9
9.3 Statistiques globales	10
10 Pipeline d'orchestration	10
11 Interface Streamlit	10
12 Tests et validation	11
12.1 Résultats de validation	11

13 Analyse critique 11

 13.1 Forces 11

 13.2 Limites 12

 13.3 Perspectives 12

A Commandes utiles 12

B Livrables principaux 12

C Note éthique 13

Conclusion 14

Bibliographie 15

Liste des tableaux

1	Architecture fonctionnelle du projet	4
2	Règles symboliques principales	6
3	Résultats observés	8
4	Échelle de criticité	8
5	Répartition du dataset	9
6	Statistiques globales	10
7	Résultats de validation	11

Introduction

La gestion foncière est un domaine sensible parce qu'elle associe droits de propriété, procédures administratives, valeurs économiques et responsabilités publiques. Dans ce contexte, plusieurs formes d'abus peuvent apparaître : concentration excessive de parcelles, revente spéculative, utilisation de prête-noms, conflits d'intérêts ou transactions circulaires. Ces situations sont difficiles à détecter automatiquement, car elles combinent des règles métier strictes et des signaux incertains.

Le projet **LandGuard Neuro-Symbolic AI** propose une architecture hybride pour répondre à cette difficulté. La logique symbolique apporte la traçabilité des règles, ProbLog permet d'exprimer l'incertitude, et PyTorch apporte une couche d'apprentissage à partir des variables du dataset. Une spécification DeepProbLog formalise ensuite le lien entre prédictions neuronales et contraintes logiques.

0.1 Objectifs

Les objectifs du projet sont les suivants :

- 1. formaliser le domaine foncier avec une taxonomie et des axiomes en logique de description ;*
- 2. implémenter une base de connaissances Prolog cohérente avec cette modélisation ;*
- 3. définir seize règles de détection symbolique ;*
- 4. modéliser l'incertitude avec ProbLog ;*
- 5. entraîner un modèle neuronal PyTorch sur les données du projet ;*
- 6. fournir une spécification DeepProbLog ;*
- 7. produire des explications lisibles pour chaque alerte ;*
- 8. orchestrer l'ensemble par un pipeline reproductible et une interface Streamlit.*

0.2 Principe éthique

Les données utilisées sont synthétiques. Les noms, parcelles, prix, dossiers et transactions ne doivent pas être interprétés comme des accusations réelles. LandGuard produit des signaux de contrôle et non des décisions définitives.

1 Contexte et problématique

Les fraudes foncières ne sont pas toujours visibles au niveau d'un dossier isolé. Un dossier peut paraître normal, alors que l'analyse des liens sociaux, des transactions et des possessions révèle une stratégie de contournement. Par exemple, un acteur peut distribuer plusieurs parcelles entre des proches, partager un téléphone ou une adresse avec un prête-nom, puis revendre les biens avec une plus-value anormale.

1.1 Limites des approches isolées

Un système expert pur est explicable mais rigide. Il fonctionne bien lorsque les conditions sont clairement définies, mais il gère mal les signaux faibles. Un modèle statistique ou neuronal peut apprendre des profils, mais ses décisions sont souvent moins transparentes. LandGuard combine donc ces deux logiques : les règles symboliques expliquent les alertes, tandis que les modules probabilistes et neuronaux quantifient le risque.

2 Architecture générale du système

L'architecture est organisée en couches complémentaires.

3 Modélisation en logique de description

3.1 Taxonomie

La modélisation du domaine repose sur cinq familles de concepts :

- **Acteur** : citoyen, agent public, promoteur, notaire ;
- **Parcelle** : parcelle urbaine, parcelle rurale ;
- **Affectation** : attribution, revente, héritage ;
- **Dossier** : dossier actif, dossier suspect ;
- **Lien social** : lien familial, professionnel ou financier.

Quelques relations de subsomption sont :

$$\textit{Citoyen} \sqsubseteq \textit{Acteur}, \quad \textit{AgentPublic} \sqsubseteq \textit{Acteur}, \quad \textit{ParcelleUrbaine} \sqsubseteq \textit{Parcelle}.$$

3.2 Rôles

Les rôles métier sont directement traduits en prédicats :

```
1 | possede(Acteur, Parcelle)
```

Couche	Fichier principal	Rôle
Logique de description	description_logic.md	Définir les concepts, rôles, axiomes et contraintes.
Base symbolique	prolog/knowledge_base.pl	Stocker les faits Prolog : acteurs, parcelles, liens, dossiers et transactions.
Règles Prolog	prolog/rules.pl	Détecter les cas d'accaparement, spéculation, conflits et réseaux.
Moteur d'inférence	prolog/inference_engine.pl	Lancer l'analyse symbolique et résumer les alertes.
Explicabilité	prolog/explainability.pl	Journaliser les règles activées et leurs justifications.
ProbLog	prolog/probabilistic_rules.pl	Calculer des probabilités de risque sur des soupçons incertains.
PyTorch	deepproblog/neural_model.py	Apprendre un modèle à partir des 200 cas du dataset.
DeepProbLog	deepproblog/deepproblog_model.py	Décrire la fusion neuro-symbolique attendue.
Pipeline	main.py	Orchestrer l'exécution et générer rapport_final.json.
Interface	streamlit_app.py	Faciliter la démonstration interactive.

TABLE 1 – Architecture fonctionnelle du projet

```

2 traite(AgentPublic, Dossier)
3 beneficiaire(Acteur, Dossier)
4 vendA(Vendeur, Acheteur, Parcelle, Date, Prix)
5 achete(Acheteur, Parcelle, Date, Prix)
6 partage_telephone(Acteur, Acteur)
7 partage_adresse(Acteur, Acteur)
8 partage_iban(Acteur, Acteur)
9 lien_familial(Acteur, Acteur)
10 lien_professionnel(Acteur, Acteur)
11 lien_financier(Acteur, Acteur)

```

3.3 Axiomes métier

Le projet formalise les scénarios de risque sous forme d'axiomes métier. Par exemple :

$$\text{Citoyen} \sqcap (\geq 4 \text{ possede.ParcelleUrbaine}) \sqsubseteq \text{AccapareurUrbain}$$

$$\text{AgentPublic} \sqcap \exists \text{traite.Dossier} \sqcap \text{beneficiaire}(\text{AgentPublic}, \text{Dossier}) \sqsubseteq \text{ConflitInteretDirect}$$

$$Acteur \sqcap \exists partageTelephone. Acteur \sqcap \exists possede.ParcelleCommune \sqsubseteq SuspectPreteNom$$

3.4 Contraintes d'intégrité

Les principales contraintes sont :

1. un agent public ne doit pas traiter son propre dossier ;
2. un citoyen ordinaire ne doit pas dépasser trois parcelles urbaines ;
3. une parcelle ne doit pas avoir deux propriétaires non justifiés ;
4. un notaire ne doit pas être bénéficiaire d'un dossier qu'il traite ;
5. un téléphone partagé avec une parcelle commune signale un prête-nom possible ;
6. une date de vente doit être postérieure à la date d'achat ;
7. un promoteur doit posséder au moins une parcelle ;
8. un dossier suspect ne doit pas rester actif sans contrôle.

4 Base de connaissances Prolog

Le fichier `prolog/knowledge_base.pl` contient les faits structurels. Il combine un noyau simple utilisé pour les tests historiques du projet et une extension contextualisée au Burkina Faso.

4.1 Exemples de faits

```

1 citoyen(adama_traore).
2 agent_public(moussa_compaore).
3 parcelle_urbaine(p_pissy_01).
4 possede(adama_traore, p_pissy_01).
5 traite(moussa_compaore, dos_bf_001).
6 beneficiaire(moussa_compaore, dos_bf_001).
7 partage_telephone(adama_traore, fatoumata_traore).
8 partage_iban(adama_traore, fatoumata_traore).
```

Ces faits permettent de déclencher des scénarios comme :

- l'accaparement familial dans le réseau Traoré ;
- l'auto-attribution de dossier par un agent public ;
- le partage de téléphone, adresse ou IBAN ;
- la transaction circulaire entre acteurs liés.

5 Raisonnement symbolique avec Prolog

5.1 Organisation des règles

Le fichier `prolog/rules.pl` contient **16 règles principales** réparties en quatre catégories.

Code	Famille	Objectif
A1	Multipropriété excessive	Détecter un acteur possédant trop de parcelles.
A2	Accaparement familial	Cumuler les possessions d'un réseau familial.
A3	Concentration urbaine	Détecter une concentration de parcelles urbaines.
A4	Monopole foncier local	Identifier un acteur dominant dans une zone.
B1	Revente ultra-rapide	Détecter une revente en moins de 90 unités temporelles.
B2	Plus-value anormale	Repérer une marge supérieure à 100%.
B3	Non-mise en valeur	Détecter l'achat sans construction depuis plus de trois ans.
B4	Transaction suspecte	Comparer le montant de vente à la valeur de marché.
C1	Auto-attribution	Identifier l'agent qui traite son propre dossier.
C2	Traitement familial	Détecter le traitement du dossier d'un proche.
C3	Favoritisme répétitif	Repérer plusieurs dossiers pour le même bénéficiaire.
C4	Conflit indirect	Détecter un lien professionnel avec le bénéficiaire.
D1	Prête-nom téléphone	Détecter téléphone partagé et parcelle commune.
D2	Prête-nom adresse	Détecter adresse partagée et parcelle commune.
D3	Transaction circulaire	Identifier un cycle de revente sur une même parcelle.
D4	IBAN partagé	Détecter un réseau financier suspect.

TABLE 2 – Règles symboliques principales

5.2 Exemple de règle

```

1 regle_d3_transaction_circulaire(X, Y, Z) :-
2     vendA(X, Y, Parcelle, Date1, _),
3     vendA(Y, Z, Parcelle, Date2, _),
4     vendA(Z, X, Parcelle, Date3, _),
5     Date1 < Date2,
6     Date2 < Date3.

```

Cette règle détecte un cycle de revente impliquant trois acteurs sur une même parcelle. Elle permet de repérer un schéma de blanchiment ou de dissimulation de bénéficiaire réel.

5.3 Moteur d'inférence

Le fichier `prolog/inference_engine.pl` orchestre l'exécution symbolique. Il analyse les citoyens, agents publics, promoteurs, cas du dataset et relations suspectes. Pour éviter une sortie trop longue, l'analyse des 200 dossiers du dataset est résumée :

```

1  Detection sur le dataset Burkina...
2    - Cas analyses: 200
3    - Cas critiques: 50
4    - Accaparements dataset: 30
5    - Speculations dataset: 40
6    - Cas limites: 20

```

6 Explicabilité XAI

L'explicabilité est assurée par `prolog/explainability.pl`. Chaque règle déclenchée enregistre :

- l'identifiant de la règle;
- les variables impliquées;
- une justification textuelle;
- une trace exploitable par le rapport XAI.

Exemple de trace :

```

1  [TRACE] Regle B1-ReventeUltraRapide declenchee
2  Variables: [adama_traore,p_pissy_01,50]
3  Justification: adama_traore a revendu la parcelle p_pissy_01
4  apres 50 jours.

```

Cette trace rend la décision auditable. L'utilisateur peut expliquer pourquoi un acteur est signalé au lieu de présenter uniquement un score.

7 Raisonnement probabiliste avec ProbLog

7.1 Clauses probabilistes

Le fichier `prolog/probabilistic_rules.pl` contient des clauses incertaines du type :

```

1  0.80::prete_nom(X, Y) :-
2      partage_telephone(X, Y),
3      X \= Y.
4
5  0.70::speculateur(X) :-

```

```

6   revente_rapide(X),
7   plus_value_anormale(X).

```

Le fichier peut être exécuté directement :

```

1   .venv/bin/problog prolog/probabilistic_rules.pl

```

7.2 Résultats observés

Requête	Probabilité
risque_fraude(abdou)	0.99685
risque_fraude(modou)	0.88750
prete_nom(abdou, ousmane)	0.93000
speculateur(abdou)	0.70000
fraude_composite(abdou)	0.69265

TABLE 3 – Résultats observés

7.3 Échelle de criticité

Probabilité	Niveau
$P < 0.30$	Faible
$0.30 \leq P < 0.60$	Moyen
$0.60 \leq P < 0.80$	Élevé
$P \geq 0.80$	Critique

TABLE 4 – Échelle de criticité

8 Couche neuronale et neuro-symbolique

8.1 Modèle PyTorch

Le module `deepproblog/neural_model.py` entraîne un réseau sur `data/dataset.csv`. Les variables utilisées sont :

- nombre de parcelles ;
- fréquence de revente ;
- plus-value ;
- nombre de liens réseau ;
- partage de téléphone ;
- âge au premier achat ;

— nombre de dossiers traités.

Le modèle prédit quatre classes :

standard, atypique, speculateur, fraudeur.

Le fichier `deepproblog/model_weights.pth` contient les poids entraînés. L'entraînement a été réalisé sur les **200 cas** du dataset.

8.2 Spécification DeepProbLog

La spécification `deepproblog/deepproblog_model_spec.pl` montre la fusion neuro-symbolique attendue :

```

1 nn(fraud_model, [X], Y,
2   [standard, atypique, speculateur, fraudeur])
3   :: neural_prediction(X, Y).
4
5 fraude(X) :-
6   neural_prediction(X, fraudeur),
7   accaparement_urbain(X).
```

La version exécutable sous SWI-Prolog, `deepproblog/deepproblog_model.pl`, sert de démonstration locale lorsque l'environnement DeepProbLog complet n'est pas disponible.

9 Dataset et analyse expérimentale

9.1 Structure du dataset

Le dataset `data/dataset.csv` contient **200 dossiers**. Il dépasse largement le volume minimal demandé dans l'énoncé.

Type de cas	Nombre	Score moyen
standard	120	0.226
speculation	20	0.790
accaparement	20	0.760
limite	20	0.655
fraude_sophistiquee	20	0.990

TABLE 5 – Répartition du dataset

9.2 Colonnes principales

Les colonnes utilisées sont :

```

1 id, nom, type, nb_parcelles, parcelles_urbaines,
```

```

2 parcelles_rurales , frequence_revente , plus_value ,
3 nb_liens_reseau , partage_telephone , partage_adresse ,
4 age_premier_achat , nb_dossiers_traites , agent_public ,
5 description , score_risque

```

9.3 Statistiques globales

Indicateur	Valeur
Nombre total de dossiers	200
Score moyen	0.455
Score minimum	0.170
Score maximum	1.000
Alertes élevées ou critiques	60
Détections intermédiaires	20

TABLE 6 – Statistiques globales

10 Pipeline d’orchestration

Le pipeline principal est `main.py`. Il réalise les étapes suivantes :

1. chargement et validation de `data/dataset.csv` ;
2. génération de `prolog/dataset_facts.pl` ;
3. exécution du moteur symbolique SWI-Prolog ;
4. exécution des requêtes ProbLog ;
5. exécution de la démonstration neuro-symbolique ;
6. génération de `rapport_final.json`.

Le script local recommandé est :

```

1 bash scripts/run_local_pipeline.sh

```

Le pipeline a été rendu plus lisible : il ne déroule plus les 200 dossiers un à un, mais affiche une synthèse des catégories détectées.

11 Interface Streamlit

Le fichier `streamlit_app.py` fournit une console de démonstration. Elle permet :

- de visualiser les indicateurs principaux du dataset ;
- de filtrer les dossiers par type et score ;
- de consulter les indices explicatifs d’un dossier ;

- de lancer le pipeline complet ;
- de télécharger les livrables ;
- de vérifier l’environnement : SWI-Prolog, ProbLog et Streamlit.

La commande recommandée est :

```
1 .venv/bin/python -m streamlit run streamlit_app.py
```

12 Tests et validation

Les tests disponibles sont :

```
1 swipl -q -s tests/test_prolog.pl
2 .venv/bin/python -m unittest tests/test_endtoend.py
3 .venv/bin/problog prolog/probabilistic_rules.pl
```

Les tests Prolog couvrent les règles historiques et les scénarios Burkina Faso ajoutés : accaparement familial, conflit familial, transaction circulaire et IBAN suspect. Les tests Python vérifient le dataset, le pipeline et les artefacts générés.

12.1 Résultats de validation

Contrôle	Résultat
Tests Prolog	OK
Tests Python end-to-end	OK
ProbLog direct	OK
Pipeline complet	OK
Rapport JSON	Généré
Interface Streamlit	Fonctionnelle

TABLE 7 – Résultats de validation

13 Analyse critique

13.1 Forces

Le projet présente plusieurs points forts :

- couverture de la logique de description, Prolog, ProbLog, PyTorch et DeepProbLog ;
- dataset de 200 cas, supérieur aux exigences minimales ;
- règles explicables et organisées en quatre familles ;
- rapport JSON consolidé ;

- interface Streamlit adaptée à la démonstration ;
- pipeline reproductible.

13.2 Limites

Les limites principales sont :

- le dataset est synthétique ;
- les probabilités ProbLog sont fixées par expertise et non calibrées sur des données réelles ;
- la couche DeepProbLog complète dépend d'un environnement académique spécifique ;
- le système fournit des alertes, pas des décisions juridiques ;
- les scénarios réels exigeraient anonymisation, audit et validation institutionnelle.

13.3 Perspectives

Les améliorations futures possibles sont :

- intégrer des données institutionnelles anonymisées ;
- calibrer les probabilités ProbLog par apprentissage ;
- formaliser la TBox dans un format OWL/RDF ;
- produire des rapports d'audit automatiques par dossier ;
- exposer le système via une API sécurisée ;
- valider les règles avec des experts fonciers.

A Commandes utiles

```
1 python3 -m venv .venv
2 .venv/bin/pip install -r requirements.txt
3
4 bash scripts/run_local_pipeline.sh
5 swipl -q -s tests/test_prolog.pl
6 .venv/bin/problog prolog/probabilistic_rules.pl
7 .venv/bin/python -m unittest tests/test_endtoend.py
8 .venv/bin/python -m streamlit run streamlit_app.py
```

B Livrables principaux

- description_logic.md
- data/dataset.csv
- prolog/knowledge_base.pl

- `prolog/rules.pl`
- `prolog/inference_engine.pl`
- `prolog/explainability.pl`
- `prolog/probabilistic_rules.pl`
- `deepproblog/neural_model.py`
- `deepproblog/deepproblog_model_spec.pl`
- `main.py`
- `streamlit_app.py`
- `rapport_final.json`
- `diagramme_concepts.pdf`
- `rapport_projet.tex`
- `rapport_projet.pdf`

C Note éthique

Les acteurs, transactions, parcelles et dossiers sont fictifs. Les villes ou zones citées servent uniquement à contextualiser les scénarios synthétiques. Toute alerte doit être interprétée comme un signal d'aide à la décision, nécessitant une vérification humaine.

Conclusion

LandGuard Neuro-Symbolic AI a permis de développer un système complet de détection de fraudes foncières combinant la Logique de Description pour la modélisation formelle du domaine avec 10 axiomes et 8 contraintes d'intégrité, un moteur d'inférence Prolog avec 16 règles de détection et un mécanisme XAI, la gestion de l'incertitude avec ProbLog et une classification du risque en 4 niveaux, ainsi qu'une fusion neuro-symbolique avec DeepProbLog et un réseau de neurones PyTorch.

Les résultats obtenus sur un dataset de 200 cas montrent une détection efficace des fraudes avec des scores de risque cohérents et des explications détaillées pour chaque alerte. Le système est capable de détecter l'accaparement, la spéculation, les prête-noms, les conflits d'intérêts et les fraudes documentaires.

Bibliographie

- Baader, F., Calvanese, D. L., McGuinness, D. L., Nardi, D., & Patel-Schneider, P. F. (Eds.). (2003). *The Description Logic Handbook : Theory, Implementation, and Applications*. Cambridge University Press.
<https://www.cambridge.org/core/books/description-logic-handbook>
- Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog*. Springer.
<https://link.springer.com/book/10.1007/978-3-642-55481-0>
- De Raedt, L., Kimmig, A., & Van den Broeck, G. (2016). *Probabilistic Programming with ProbLog*. In : K. Kersting et al. (eds.), *Inductive Logic Programming and Machine Learning*. Springer.
<https://dtai.cs.kuleuven.be/problog/>
- Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., & De Raedt, L. (2018). *DeepProbLog : Neural Probabilistic Logic Programming*. In *Advances in Neural Information Processing Systems (NeurIPS 2018)*.
<https://arxiv.org/abs/1805.10872>
- Paszke, A., et al. (2019). *PyTorch : An Imperative Style, High-Performance Deep Learning Library*. In *Advances in Neural Information Processing Systems (NeurIPS 2019)*.
<https://arxiv.org/abs/1912.01703>
- Russell, S., & Norvig, P. (2021). *Artificial Intelligence : A Modern Approach (4th ed.)*. Pearson.
<https://aima.cs.berkeley.edu>
- Van Harmelen, F., Lifschitz, V., & Porter, B. (Eds.). (2008). *Handbook of Knowledge Representation*. Elsevier.
<https://www.sciencedirect.com/bookseries/handbooks-in-artificial-intelligence-and-applications/vol/3>